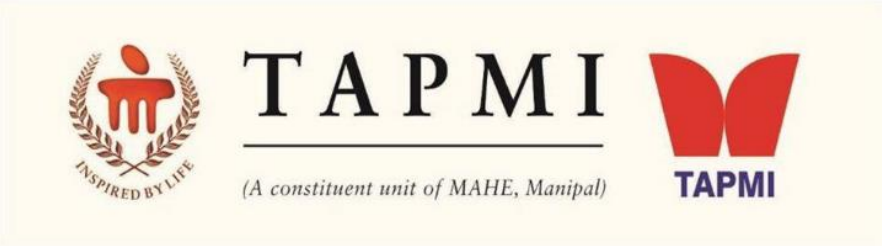
CYBERSECURITY PROJECT REPORT

ANALYSIS AND SOLUTION



NAME: POOJITHA K

ID: 251620140002

Overview

The SecureBank portal was built in two versions. The first ('vulnerable') is the AI-built first draft. It was then audited with a second, independent tool set - Bandit (which reads the code) and pip-audit (which checks the libraries) - plus a hands-on review and a working attack for each flaw. The second version ('fixed') closes every problem. Six issues were found: five are live in the vulnerable app. Bandit drops from four findings to zero, the automated test suite passes, and every attack that works in the first version is blocked in the second. All screenshots are collected at the end of this report.

The six vulnerabilities

Each vulnerability is summarised below in plain words: what it is, how it works, and the real-world impact. The prevention table that follows shows, for every one, how banks generally defend against it and how we fixed it in this project.

1. SQL Injection (login bypass)

Category: Confidentiality / Input Validation (OWASP A03)

Typing special characters into the username box turns the text into part of the database command. Entering x' OR '1'='1' -- makes the login always true and

comments out the password check, so the attacker logs in with no real password. Impact: full account or admin takeover without knowing a password.

2. Broken Access Control (IDOR)

Category: Confidentiality / Authorization (OWASP A01)

The app fetched an account by the id in the URL with no ownership check, so changing /account/1 to /account/2 showed another customer's balance, account number and full statement. Impact: a mass privacy breach - any logged-in user could read every customer's account.

3. Weak Authentication (single factor, no lockout)

Category: Authentication / MFA (OWASP A07)

Login needed only a password and never blocked repeated wrong tries (it showed 'Failed attempts: 1,2,3...' but never locked). Impact: account takeover by password guessing or by reusing a leaked password; the admin account is the biggest prize.

4. CSRF - the 'lottery link'

Category: Authorization / CSRF (OWASP A01)

A fake 'Claim your Rs 1,00,000 lucky draw' link opened an attacker page that silently auto-submitted a transfer using the user's logged-in session; Rs 500 left the account with no confirmation. Impact: money stolen directly, with the user never clicking anything on the real bank site.

5. Open Redirect leading to Card Phishing

Category: Phishing / Redirect (OWASP A03)

The login trusted a ?next= value and redirected to it exactly. A 'Re-verify now' link logged the user into the real bank, then bounced them to a look-alike page that asked for their card number and ATM PIN. Impact: theft of card and PIN, which an attacker can use to drain the account.

How it is prevented: bank practice vs our fix

For each vulnerability, the table shows how a real bank generally defends against it and the specific fix applied in this project.

Vulnerability	How banks generally prevent it	How we fixed it here
SQL Injection	Parameterized queries; input validation; least-privilege DB	All queries parameterized (input is sent as data, never

	accounts; code scanning in the build pipeline.	as command); passwords stored as salted scrypt hashes.
Broken Access Control (IDOR)	Server-side ownership/permission checks on every record; never rely on hard-to-guess ids; log blocked attempts.	Server-side ownership check on every account; a non-owner now gets a '403 Access denied' page.
Weak Authentication	Multi-factor authentication (OTP); account lockout / rate limiting; breached-password checks; new-device alerts.	Password + SMS OTP (RBI model) required; account locks after 5 failed attempts.
CSRF (fake links)	CSRF tokens on every state-changing form; SameSite cookies; re-authenticate for large transfers; CSP to stop injected links.	Flask-WTF CSRF token on every money/action form; a forged request has no token and is rejected (HTTP 400).
Open Redirect / Phishing	Redirect allow-list; HTTPS + HSTS + verifiable certificate; SPF/DKIM/DMARC email anti-spoofing; look-alike domain takedown; user awareness.	Redirect allowed only to the bank's own pages (allow-list); the fake page is refused; a fraud-awareness banner added.

Stopping fake links and fake websites

The two attacks users ask about most are the fake 'lottery' link (CSRF) and the fake bank website (phishing). The flowcharts below show how each attack flows and, in green, exactly where each control stops it.

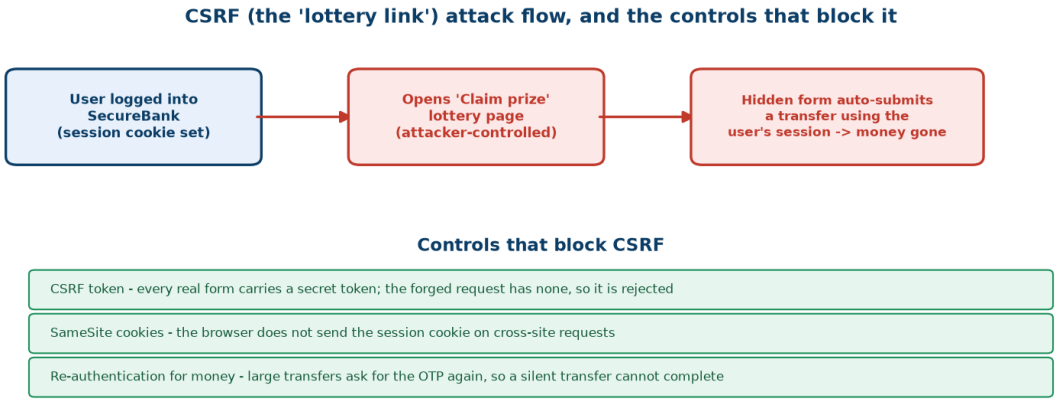


Figure B. The 'lottery link' (CSRF) attack and the controls that block it.

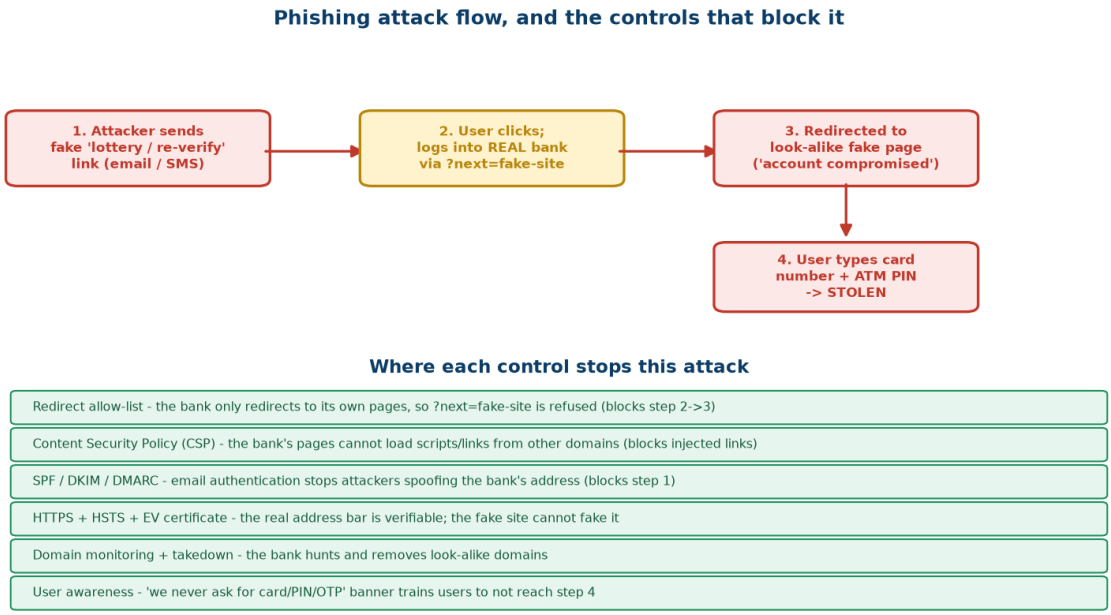


Figure C. The phishing (fake website) attack and the controls that block it.

Fake links and fake websites

- **Output escaping + Content Security Policy (CSP)** — block scripts and links injected into the site.
- **CSRF tokens + SameSite cookies** — block forged requests from other sites.
- **HTTPS/TLS + HSTS** — protect data in transit and make the real address bar verifiable.
- **SPF / DKIM / DMARC + user awareness** — stop spoofed emails carrying fake links.
- **Redirect allow-list + domain takedowns** — keep users on the real site and remove look-alike domains.

Conclusion. All six issues were found with independent tools and a manual review, and all six are fixed. Bandit drops from four findings to zero, and every attack is now blocked.

Appendix - Screenshots

Security-tool evidence

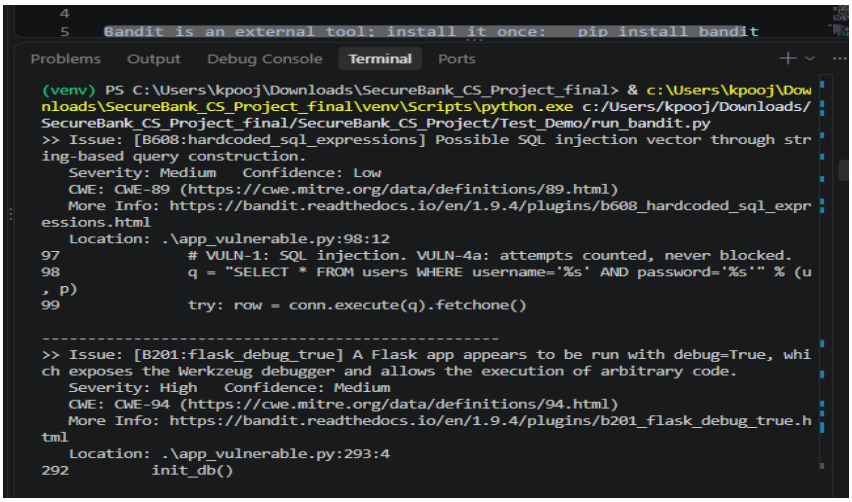


Figure 1. Bandit scan of the vulnerable app - findings present.

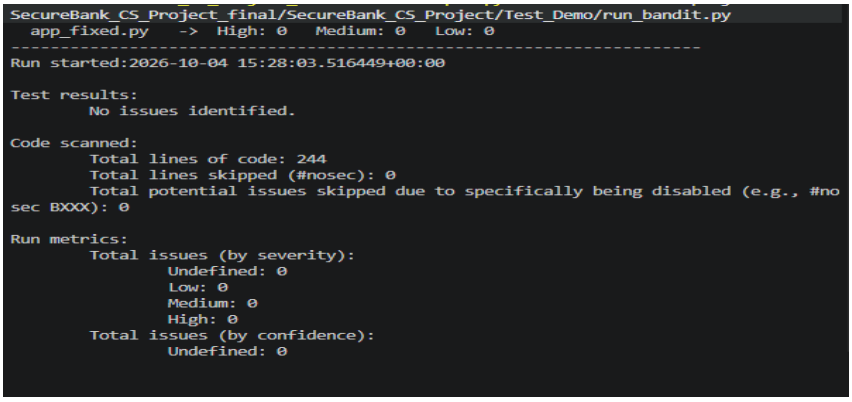


Figure 2. Bandit scan of the fixed app - zero issues.

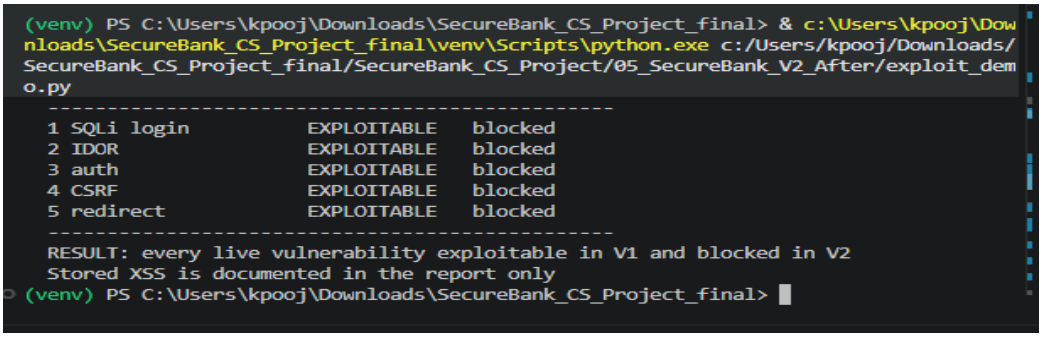


Figure 3. The attack demo - exploitable before, blocked after.

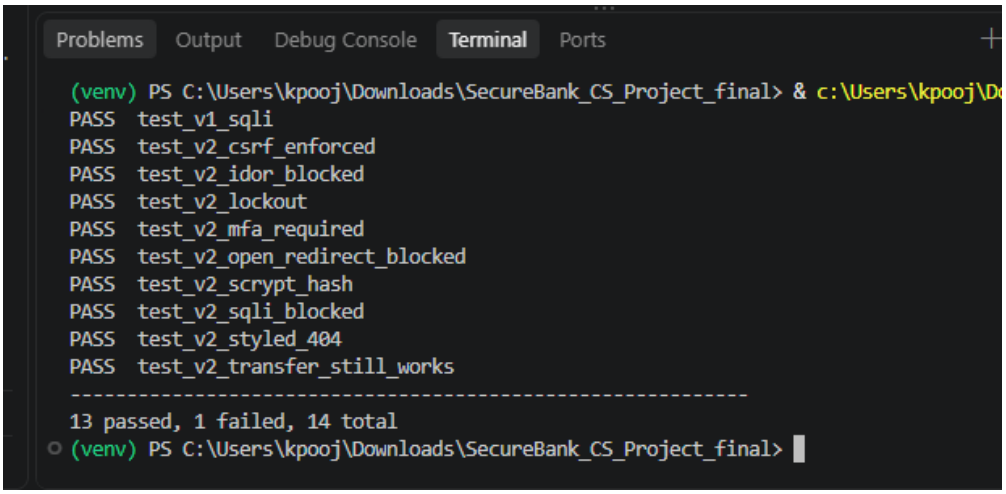


Figure 4. The automated test suite - all checks pass.

The vulnerabilities in the browser

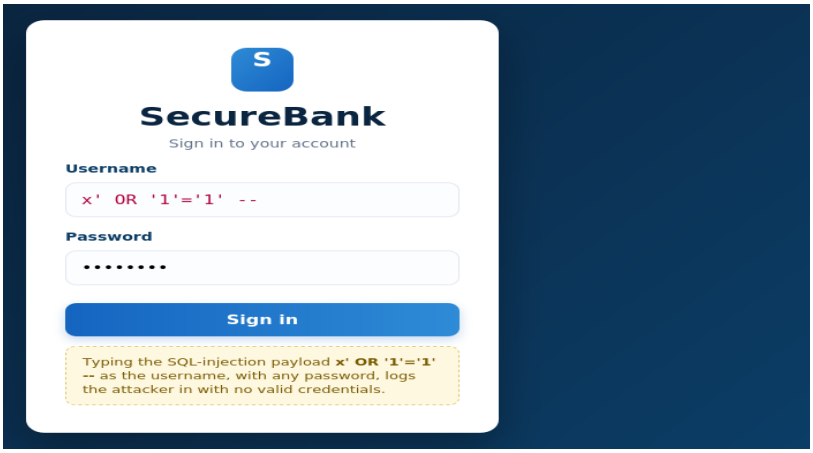


Figure 5. SQL injection - logging in with a payload instead of a password.

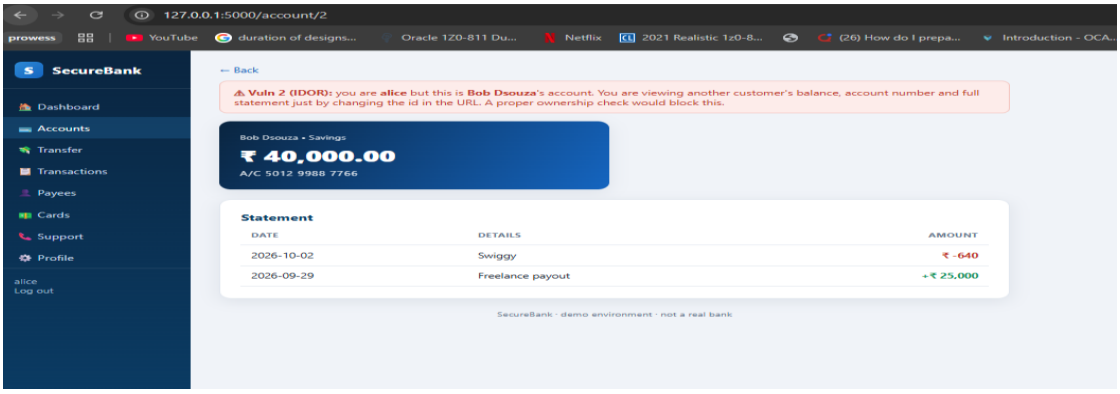


Figure 6. IDOR - Alice changes the URL to /account/2 and sees Bob's account.

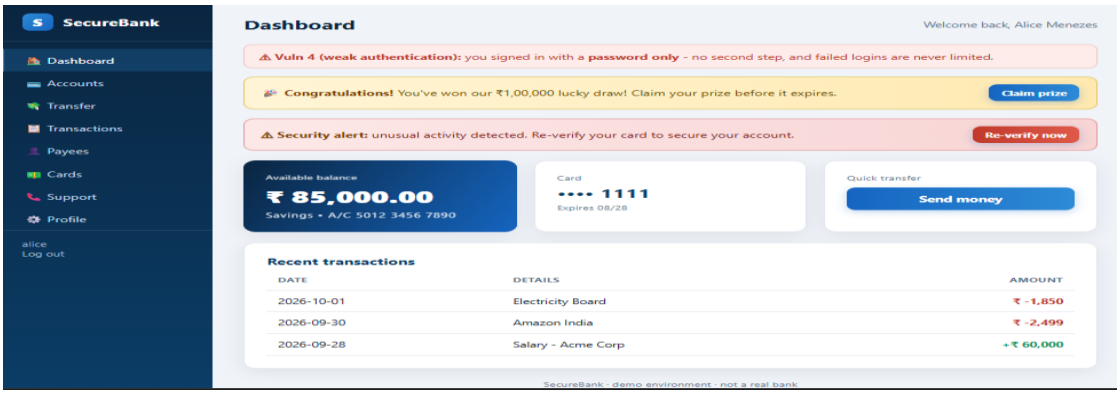


Figure 7. Weak authentication - the dashboard warns login used a password only.

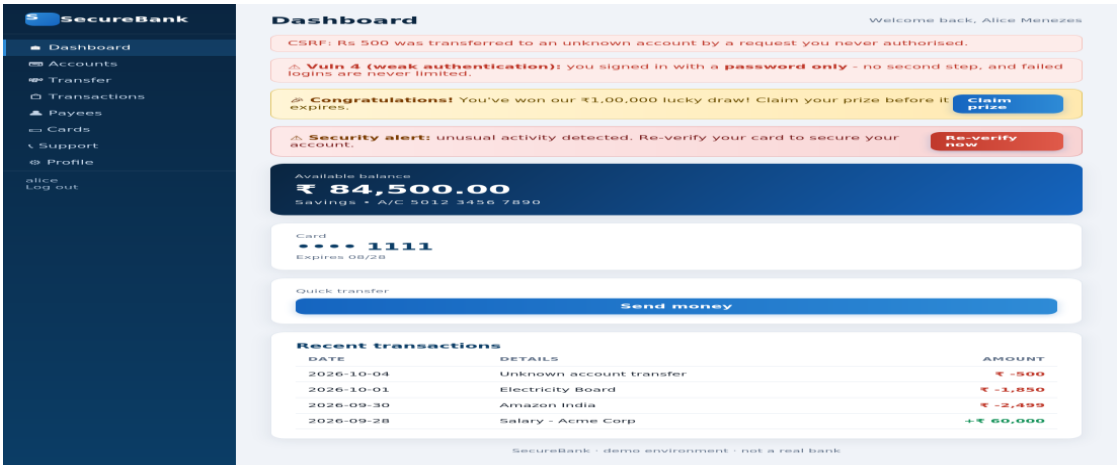


Figure 8. CSRF - after a fake prize link, Rs 500 goes to an unknown account.

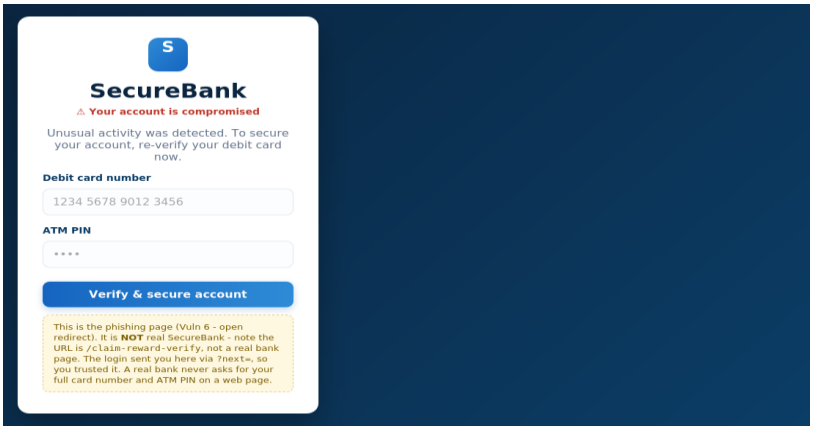


Figure 9. Phishing - the fake page capturing card number and ATM PIN.

The fixes in action

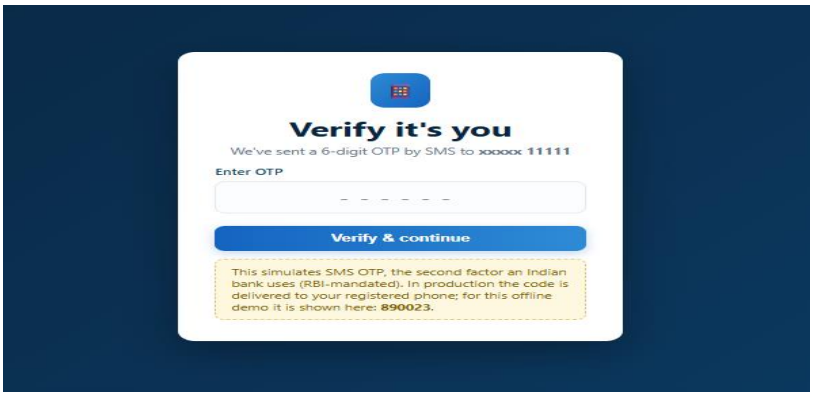


Figure 10. Two-factor login - an SMS OTP is now required.

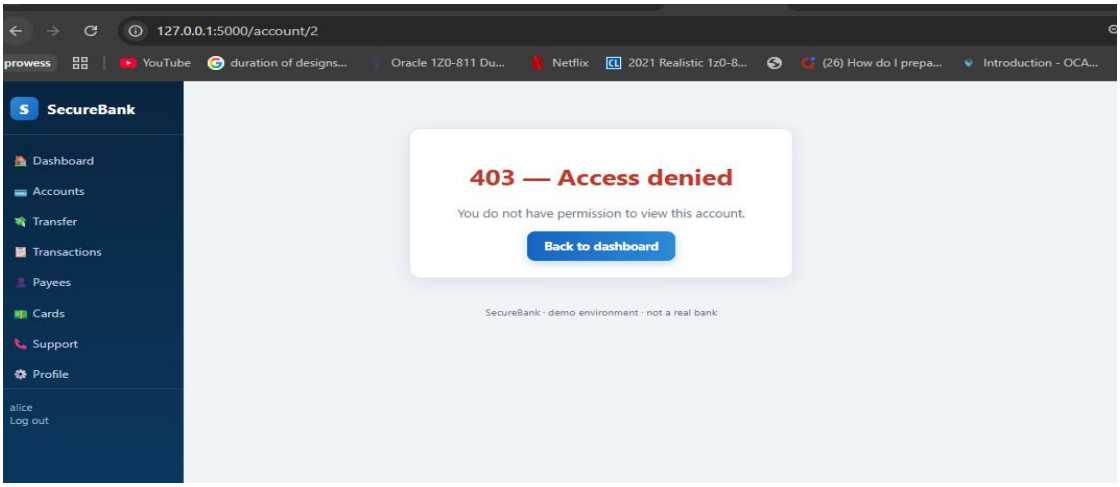


Figure 11. IDOR now refused with a '403 Access denied' page.

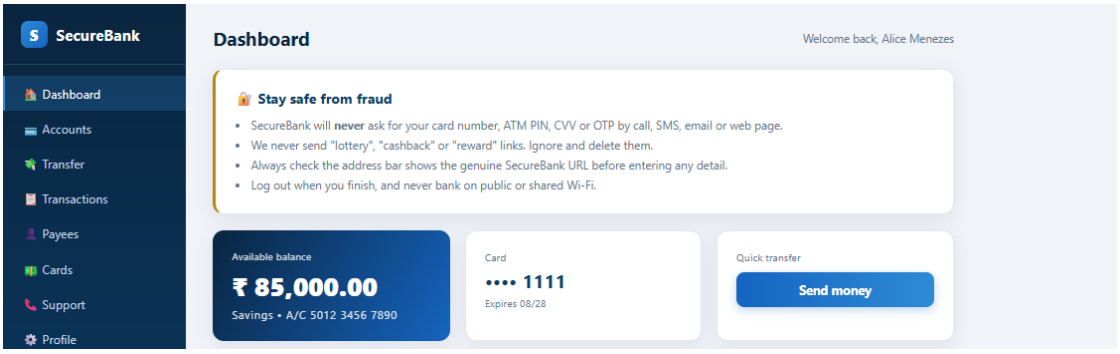


Figure 12. The fixed dashboard shows a fraud-awareness card instead of attacker bait.